Proyecto - Sistema de Inventario y Facturación para Comercios Locales de Cartago

Brayan Euclides Cardenas Blandon

Corporación de Estudios Tecnológicos del Norte del Valle

Cartago, Valle del Cauca

Tabla de contenido

1. INTRODUCCIÓN 4
1.1 Contexto de la Problemática 4
1.2 Objetivos del Sistema 4
Objetivos Específicos 5
2. MODELO ENTIDAD-RELACIÓN (MER) 6
2.1 Diagrama Entidad-Relación 6
2.2 Descripción de las Entidades y sus Atributos 6
Entidad: categorias 6
Entidad: proveedores 6
Entidad: clientes 7
Entidad: productos 7
Entidad: facturas 7
Entidad: detalle_factura 7
Entidad: alertas_stock 8
Entidad: auditoria_precios 8
2.3 Explicación de las Relaciones 8
2.4 Restricciones de Integridad 9
3. ESQUEMA RELACIONAL IMPLEMENTADO 10
3.1 Listado de Tablas 10
3.2 Justificación de la Elección de Tipos de Datos 10
4. PROCEDIMIENTOS ALMACENADOS DESARROLLADOS 11
4.1 Procedimiento registrar_venta 11
Parámetros: 11
Ejemplo de ejecución: 11
4.2 Procedimiento consultar_stock_producto 11
Parámetros: 11
Ejemplo de ejecución: 11
4.3 Procedimiento calcular_ganancia_periodo 12
Parámetros: 12
Ejemplo de ejecución: 12
4.4 Procedimiento listar_productos_por_categoria 12

Parámetros:.....12
Ejemplo de ejecución:.....12
4.5 Procedimiento aplicar_descuento.....12
Parámetros:.....12
Ejemplo de ejecución:.....13
5. TRIGGERS IMPLEMENTADOS.....14
5.1 Trigger trg_alerta_stock_minimo.....14
Ejemplo de funcionamiento.....14
5.2 Trigger trg_auditar_precio.....14
Ejemplo de funcionamiento.....14
5.3 Trigger trg_actualizar_total_factura.....14
Ejemplo de funcionamiento.....15
6. EVIDENCIA DEL ENTORNO FUNCIONANDO.....16
6.1 Contenedores Docker Activos.....16
6.2 Acceso a phpMyAdmin.....16
6.3 Conexión desde MySQL Workbench.....17
6.4 Ejecución de Procedimientos y Triggers.....17
7. CONCLUSIONES Y TRABAJO FUTURO.....24
7.1 Aprendizajes Obtenidos.....24
8. REFERENCIAS BIBLIOGRÁFICAS.....25

1. INTRODUCCIÓN

1.1 Contexto de la Problemática

En el municipio de Cartago, Valle del Cauca, los pequeños y medianos comercios locales enfrentan serias dificultades en la gestión de sus inventarios y en el proceso de facturación. La mayoría de estos negocios aún utilizan métodos manuales como cuadernos, hojas de cálculo en Excel o sistemas básicos que no ofrecen un control integral de sus operaciones. Esta situación genera múltiples problemáticas que afectan directamente la rentabilidad y sostenibilidad de estos negocios.

Entre las principales dificultades identificadas se encuentran: la pérdida de ventas por falta de conocimiento en tiempo real sobre la disponibilidad de productos, errores frecuentes en el cálculo de totales e impuestos en las facturas, ausencia de alertas cuando un producto está por agotarse, imposibilidad de rastrear el historial de cambios de precios, y la falta de reportes que permitan tomar decisiones informadas sobre qué productos son más rentables y cuáles deben descontinuarse.

Según datos de la Cámara de Comercio de Cartago, aproximadamente el 70% de los comercios locales son microempresas que no cuentan con un sistema informático adecuado para su gestión. Esto representa una oportunidad significativa para aplicar soluciones tecnológicas que mejoren su competitividad y eficiencia operativa.

1.2 Objetivos del Sistema

Objetivo General:

Diseñar e implementar una base de datos relacional que permita gestionar de manera eficiente el inventario, las ventas y la facturación de los comercios locales de Cartago, aplicando los conceptos de normalización, procedimientos almacenados y triggers vistos en la asignatura Bases de Datos II.

Objetivos Específicos:

1. **Diseñar un modelo entidad-relación normalizado hasta la Tercera Forma Normal (3FN)** que elimine la redundancia de datos y garantice la integridad referencial, permitiendo un almacenamiento eficiente y consistente de la información del inventario, clientes, proveedores y facturación.

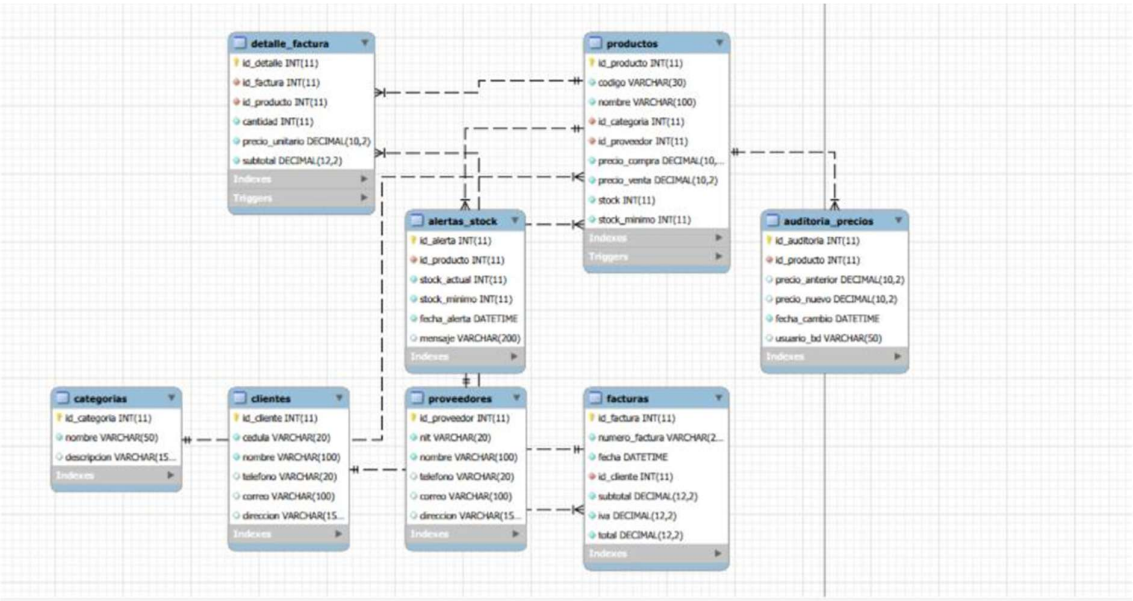
2. **Implementar procedimientos almacenados** que automaticen las operaciones críticas del negocio, como el registro de ventas con descuento automático de stock, la consulta de disponibilidad de productos, el cálculo de ganancias por período y la aplicación de descuentos, reduciendo así el margen de error humano y mejorando la eficiencia operativa.

3. **Desarrollar triggers que automaticen la validación y auditoría de datos**, incluyendo alertas automáticas cuando el stock de un producto cae por debajo del mínimo establecido, el registro histórico de cambios de precios para análisis de tendencias, y la actualización automática de totales en las facturas al agregar productos.

2. MODELO ENTIDAD-RELACIÓN (MER)

2.1 Diagrama Entidad-Relación

El modelo entidad-relación del sistema de inventario y facturación está compuesto por ocho entidades principales interconectadas mediante relaciones de uno a muchos (1:N). El diagrama fue elaborado en MySQL Workbench y posteriormente exportado a la base de datos MariaDB mediante la herramienta Forward Engineer.



2.2 Descripción de las Entidades y sus Atributos

Entidad: categorias

Almacena las categorías de productos disponibles en el inventario. Sus atributos son: id_categoria (INT, PK, AUTO_INCREMENT), nombre (VARCHAR(50), NOT NULL, UNIQUE) y descripcion (VARCHAR(150)). Permite clasificar los productos para facilitar su búsqueda y organización.

Entidad: proveedores

Contiene la información de los proveedores que suministran los productos. Atributos: id_proveedor (INT, PK, AUTO_INCREMENT), nit (VARCHAR(20), NOT NULL, UNIQUE), nombre (VARCHAR(100), NOT

NULL), telefono (VARCHAR(20)), correo (VARCHAR(100))

y direccion (VARCHAR(150)).

Entidad: clientes

Registra los datos de los clientes que realizan compras. Atributos: id_cliente (INT, PK, AUTO_INCREMENT), cedula (VARCHAR(20), NOT NULL, UNIQUE), nombre (VARCHAR(100), NOT NULL), telefono (VARCHAR(20)), correo (VARCHAR(100)) y direccion (VARCHAR(150)).

Entidad: productos

Es la entidad central del sistema. Almacena todos los productos del inventario. Atributos: id_producto (INT, PK, AUTO_INCREMENT), codigo (VARCHAR(30), NOT NULL, UNIQUE), nombre (VARCHAR(100), NOT NULL), id_categoria (INT, FK → categorias), id_proveedor (INT, FK → proveedores), precio_compra (DECIMAL(10,2), NOT NULL), precio_venta (DECIMAL(10,2), NOT NULL), stock (INT, NOT NULL) y stock_minimo (INT, NOT NULL).

Entidad: facturas

Almacena las cabeceras de las facturas generadas por las ventas. Atributos: id_factura (INT, PK, AUTO_INCREMENT), numero_factura (VARCHAR(20), NOT NULL, UNIQUE), fecha (DATETIME, NOT NULL, DEFAULT CURRENT_TIMESTAMP), id_cliente (INT, FK → clientes), subtotal (DECIMAL(12,2), NOT NULL), iva (DECIMAL(12,2), NOT NULL) y total (DECIMAL(12,2), NOT NULL).

Entidad: detalle_factura

Contiene los productos específicos incluidos en cada factura. Atributos: id_detalle (INT, PK, AUTO_INCREMENT), id_factura (INT, FK → facturas), id_producto (INT, FK →

productos), cantidad (INT, NOT NULL), precio_unitario (DECIMAL(10,2), NOT NULL) y subtotal (DECIMAL(12,2), NOT NULL).

Entidad: alertas_stock

Registra las alertas generadas automáticamente por el trigger cuando el stock de un producto está por debajo del mínimo. Atributos: id_alerta (INT, PK, AUTO_INCREMENT), id_producto (INT, FK → productos), stock_actual (INT, NOT NULL), stock_minimo (INT, NOT NULL), fecha_alerta (DATETIME, NOT NULL) y mensaje (VARCHAR(200)).

Entidad: auditoria_precios

Almacena el histórico de cambios de precios de los productos. Atributos: id_auditoria (INT, PK, AUTO_INCREMENT), id_producto (INT, FK → productos), precio_anterior (DECIMAL(10,2)), precio_nuevo (DECIMAL(10,2)), fecha_cambio (DATETIME, NOT NULL) y usuario_bd (VARCHAR(50)).

2.3 Explicación de las Relaciones

Relaciones identificativas: No se presentan en este modelo, ya que ninguna tabla depende de otra para su identificación única. Todas las tablas tienen su propia llave primaria autoincremental.

Relaciones no identificativas: Todas las relaciones del modelo son no identificativas, representadas con líneas punteadas en el diagrama MER:

1. **categorias → productos (1:N):** Una categoría puede tener muchos productos, pero un producto pertenece a una sola categoría.
2. **proveedores → productos (1:N):** Un proveedor puede suministrar muchos productos, pero un producto tiene un solo proveedor.
3. **clientes → facturas (1:N):** Un cliente puede realizar muchas compras, pero cada factura pertenece a un solo cliente.

4. **facturas → detalle_factura (1:N):** Una factura puede tener muchos detalles (productos), pero cada detalle pertenece a una sola factura.

5. **productos → detalle_factura (1:N):** Un producto puede aparecer en muchos detalles de factura.

6. **productos → alertas_stock (1:N):** Un producto puede generar múltiples alertas de stock.

7. **productos → auditoria_precios (1:N):** Un producto puede tener múltiples registros de auditoría de precios.

2.4 Restricciones de Integridad

Llaves Primarias (PK): Cada tabla tiene su propia PK definida con AUTO_INCREMENT para garantizar unicidad.

Llaves Foráneas (FK): Se definieron 9 llaves foráneas que garantizan la integridad referencial entre las tablas.

Restricciones UNIQUE: Se aplican en categorias.nombre, proveedores.nit, clientes.cedula, productos.codigo y facturas.numero_factura para evitar duplicados.

Restricciones NOT NULL: Se aplican en campos obligatorios como nombres, precios, stock y fechas.

3. ESQUEMA RELACIONAL IMPLEMENTADO

3.1 Listado de Tablas

Tabla	Columnas	Tipos de Datos	Restricciones
categorias	id_categoria, nombre, descripcion	INT, VARCHAR(50), VARCHAR(150)	PK, NOT NULL, UNIQUE
proveedores	id_proveedor, nit, nombre, telefono, correo, direccion	INT, VARCHAR(20), VARCHAR(100), VARCHAR(20), VARCHAR(100), VARCHAR(150)	PK, NOT NULL, UNIQUE
clientes	id_cliente, cedula, nombre, telefono, correo, direccion	INT, VARCHAR(20), VARCHAR(100), VARCHAR(20), VARCHAR(100), VARCHAR(150)	PK, NOT NULL, UNIQUE
productos	id_producto, codigo, nombre, id_categoria, id_proveedor, precio_compra, precio_venta, stock, stock_minimo	INT, VARCHAR(30), VARCHAR(100), INT, INT, DECIMAL(10,2), DECIMAL(10,2), INT, INT	PK, FK, NOT NULL, UNIQUE
facturas	id_factura, numero_factura, fecha, id_cliente, subtotal, iva, total	INT, VARCHAR(20), DATETIME, INT, DECIMAL(12,2), DECIMAL(12,2), DECIMAL(12,2)	PK, FK, NOT NULL, UNIQUE
detalle_factura	id_detalle, id_factura, id_producto, cantidad, precio_unitario, subtotal	INT, INT, INT, INT, DECIMAL(10,2), DECIMAL(12,2)	PK, FK, NOT NULL
alertas_stock	id_alerta, id_producto, stock_actual, stock_minimo, fecha_alerta, mensaje	INT, INT, INT, INT, DATETIME, VARCHAR(200)	PK, FK, NOT NULL
auditoria_precios	id_auditoria, id_producto, precio_anterior, precio_nuevo, fecha_cambio, usuario_bd	INT, INT, DECIMAL(10,2), DECIMAL(10,2), DATETIME, VARCHAR(50)	PK, FK

3.2 Justificación de la Elección de Tipos de Datos

INT para identificadores: Se utiliza INT con AUTO_INCREMENT para las llaves primarias, permitiendo hasta 2.147.483.647 registros, suficiente para cualquier comercio local.

VARCHAR para textos: Se usa VARCHAR en lugar de CHAR para optimizar el almacenamiento, ya que los nombres y direcciones tienen longitudes variables.

DECIMAL para precios: Se eligió DECIMAL(10,2) y DECIMAL(12,2) para valores monetarios porque evita errores de redondeo que ocurren con FLOAT o DOUBLE, garantizando precisión exacta en centavos.

DATETIME para fechas: Permite almacenar fecha y hora exacta, útil para auditoría y trazabilidad de operaciones.

UNIQUE para códigos: Se aplica en campos como NIT, cédula, código de producto y número de factura para evitar duplicados que causarían confusión.

4. PROCEDIMIENTOS ALMACENADOS DESARROLLADOS

4.1 Procedimiento registrar_venta

Propósito: Registra una venta completa, verificando stock disponible, creando la factura, insertando el detalle y descontando el stock del producto.

Parámetros:

- IN p_id_cliente (INT): ID del cliente que realiza la compra
- IN p_id_producto (INT): ID del producto vendido
- IN p_cantidad (INT): Cantidad de unidades vendidas
- OUT p_mensaje (VARCHAR): Mensaje de confirmación o error
- OUT p_total (DECIMAL): Total de la factura con IVA

Código SQL: [Ver archivo sql/procedimientos.sql]

Ejemplo de ejecución:

```
CALL registrar_venta(1, 1, 1, @mensaje, @total);
SELECT @mensaje, @total;
```

Resultado: Mensaje: "Venta registrada. Factura: FAC-011", Total: 2975000.00

4.2 Procedimiento consultar_stock_producto

Propósito: Consulta el stock y precio de un producto específico por su código.

Parámetros:

- IN p_codigo (VARCHAR): Código del producto
- OUT p_nombre (VARCHAR): Nombre del producto
- OUT p_stock (INT): Stock actual
- OUT p_precio (DECIMAL): Precio de venta

Ejemplo de ejecución:

```
CALL consultar_stock_producto('P001', @nombre, @stock, @precio);
SELECT @nombre, @stock, @precio;
```

Resultado: Portatil HP Pavilion, 14, 2500000.00

4.3 Procedimiento calcular_ganancia_periodo

Propósito: Calcula la ganancia total obtenida en un rango de fechas, considerando la diferencia entre precio de venta y precio de compra.

Parámetros:

- IN p_fecha_inicio (DATE): Fecha inicial del período
- IN p_fecha_fin (DATE): Fecha final del período
- OUT p_ganancia_total (DECIMAL): Ganancia total calculada

Ejemplo de ejecución:

```
CALL calcular_ganancia_periodo('2024-09-01', '2024-09-30', @ganancia);
SELECT @ganancia;
```

Resultado: 3300000.00

4.4 Procedimiento listar_productos_por_categoria

Propósito: Lista todos los productos pertenecientes a una categoría específica.

Parámetros:

- IN p_categoria (VARCHAR): Nombre de la categoría

Ejemplo de ejecución:

```
CALL listar_productos_por_categoria('Computadores');
```

Resultado: Tabla con los productos de la categoría.

4.5 Procedimiento aplicar_descuento

Propósito: Aplica un porcentaje de descuento a un precio, demostrando el uso del parámetro INOUT.

Parámetros:

- INOUT p_precio (DECIMAL): Precio original que será modificado
- IN p_porcentaje (DECIMAL): Porcentaje de descuento a aplicar

Ejemplo de ejecución:

```
SET @precio = 1000000;

CALL aplicar_descuento(@precio, 10);

SELECT @precio;
```

Resultado: 900000.00 (se aplicó 10% de descuento)

5. TRIGGERS IMPLEMENTADOS

5.1 Trigger trg_alerta_stock_minimo

Evento: AFTER UPDATE en la tabla productos

Utilidad: Genera automáticamente una alerta cuando el stock de un producto cae por debajo del mínimo establecido, permitiendo al comerciante tomar acciones oportunas para reabastecer el inventario.

Código SQL: [Ver archivo sql/triggers.sql]

Ejemplo de funcionamiento:

UPDATE productos SET stock = 2 WHERE id_producto = 1;

SELECT * FROM alertas_stock ORDER BY id_alerta DESC LIMIT 1;

Resultado: Alerta generada: "ALERTA: Portatil HP Pavilion tiene stock bajo (2 unidades). Minimo requerido: 3"

5.2 Trigger trg_auditar_precio

Evento: BEFORE UPDATE en la tabla productos

Utilidad: Registra en una tabla de auditoría todos los cambios de precio que se realicen en los productos, permitiendo analizar la evolución de precios y detectar posibles errores o cambios no autorizados.

Ejemplo de funcionamiento:

UPDATE productos SET precio_venta = 2700000 WHERE id_producto = 1;

SELECT * FROM auditoria_precios ORDER BY id_auditoria DESC LIMIT 1;

Resultado: Registro: precio_anterior=2500000, precio_nuevo=2700000, usuario_bd=root@172.20.0.1

5.3 Trigger trg_actualizar_total_factura

Evento: AFTER INSERT en la tabla detalle_factura

Utilidad: Actualiza automáticamente los totales (subtotal, IVA y total) de la factura cada vez que se agrega un nuevo producto al detalle, garantizando la consistencia de los datos sin intervención manual.

Ejemplo de funcionamiento:

```
SELECT * FROM facturas ORDER BY id_factura DESC LIMIT 1;
```

Resultado: Factura FAC-011 con subtotal, IVA y total correctamente calculados.

6. EVIDENCIA DEL ENTORNO FUNCIONANDO

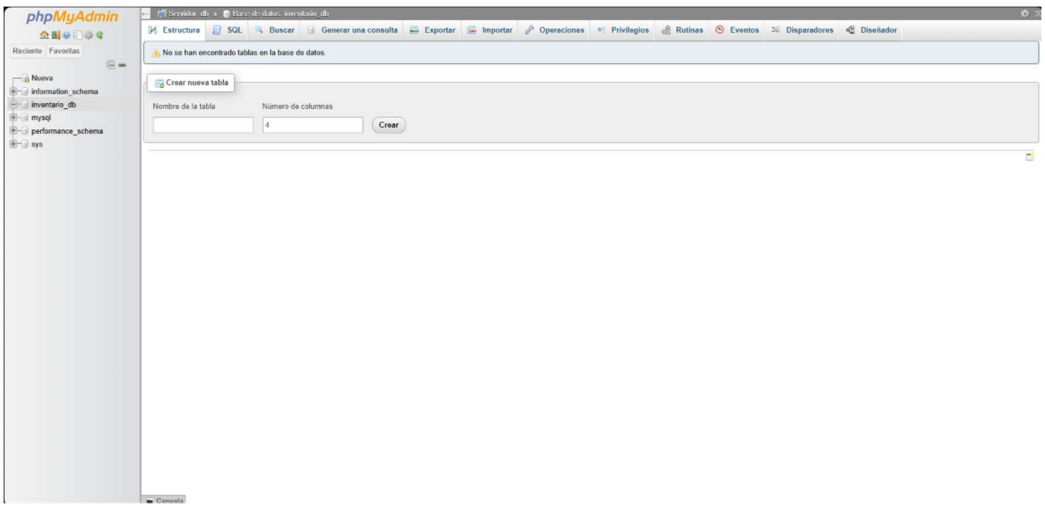
6.1 Contenedores Docker Activos

```
aleja@DESKTOP-H2IMHOI:~/BDII/proyecto-bd2$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS        PORTS                                                  NAMES
7f21b1c34a75   proyecto-bd2-app               "docker-php-entrypoi…"   2 minutes ago  Up 2 minutes  0.0.0.0:8100->80/tcp, [::]:8100->80/tcp                proyecto_app
3fb4fd17ba7a   phpmyadmin/phpmyadmin:latest   "/docker-entrypoint.…"   2 minutes ago  Up 2 minutes  0.0.0.0:8101->80/tcp, [::]:8101->80/tcp                proyecto_phpmyadmin
e6b1261a47ce   mariadb:10.8                   "docker-entrypoint.s…"   2 minutes ago  Up 2 minutes  0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp            proyecto_mariadb
5fded03d810a   parciall-bbdd-app              "docker-php-entrypoi…"   5 days ago     Up 5 days     0.0.0.0:8090->80/tcp, [::]:8090->80/tcp                parcial_app
7e8b78e4b10d   phpmyadmin/phpmyadmin:latest   "/docker-entrypoint.…"   5 days ago     Up 5 days     0.0.0.0:8091->80/tcp, [::]:8091->80/tcp                parcial_phpmyadmin
4447f9733417   mariadb:10.8                   "docker-entrypoint.s…"   5 days ago     Up 5 days     0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp            parcial_mariadb
fcfd1937cc0d   claseii-app                    "docker-php-entrypoi…"   4 weeks ago    Up 4 weeks    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp                bases_de_datos
3ff438746d14   phpmyadmin/phpmyadmin:latest   "/docker-entrypoint.…"   4 weeks ago    Up 4 weeks    0.0.0.0:8081->80/tcp, [::]:8081->80/tcp                bbdd_phpmyadmin
a13884295430   mariadb:10.8                   "docker-entrypoint.s…"   4 weeks ago    Up 4 weeks    0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp            bbdd_mariadb
```

Se observan los tres contenedores corriendo correctamente:

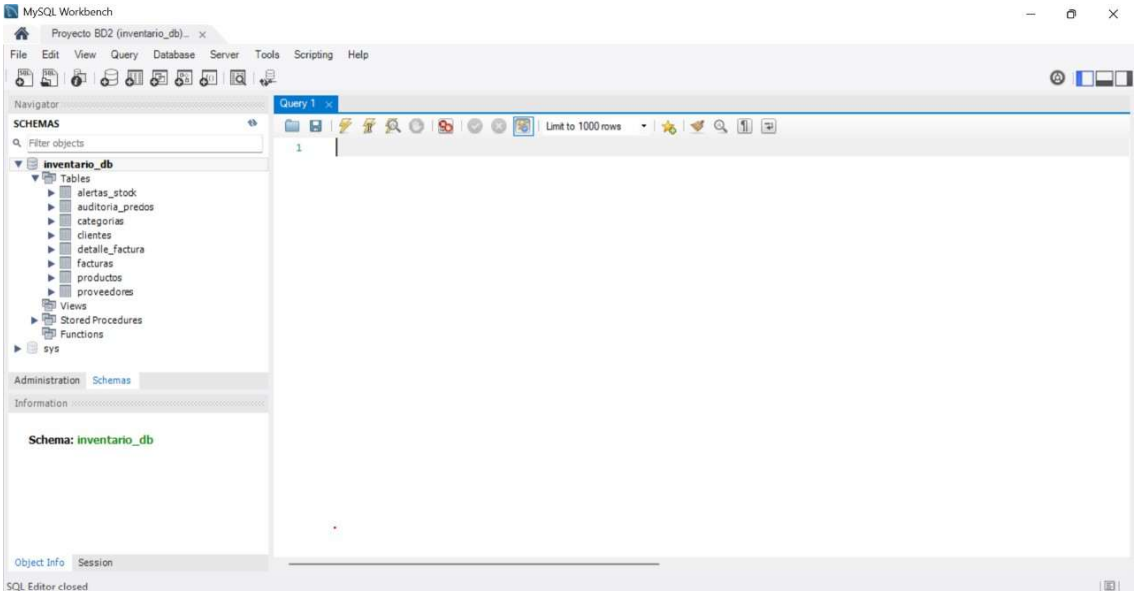
- proyecto_app en el puerto 8100
- proyecto_phpmyadmin en el puerto 8101
- proyecto_mariadb en el puerto 3308

6.2 Acceso a phpMyAdmin



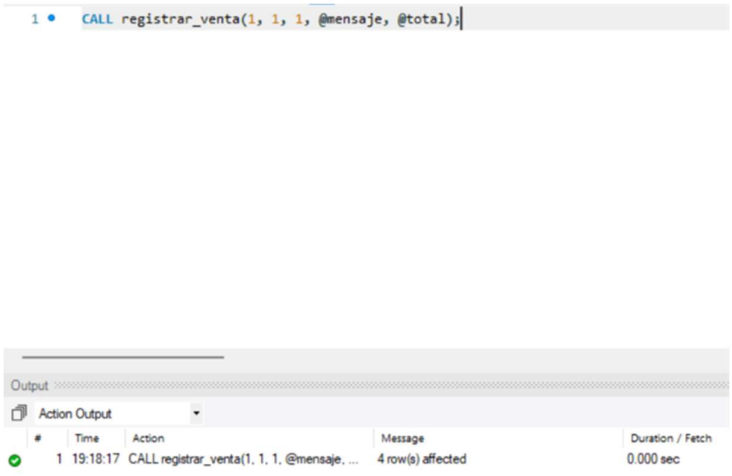
Se accede correctamente a phpMyAdmin mediante http://localhost:8101, donde se visualizan las 8 tablas de la base de datos inventario_db.

6.3 Conexión desde MySQL Workbench



MySQL Workbench conectado exitosamente al puerto 3308, mostrando las 8 tablas del proyecto y el diagrama EER.

6.4 Ejecución de Procedimientos y Triggers



```
1   SELECT @mensaje AS Mensaje, @total AS Total;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Mensaje	Total
Venta registrada. Factura: FAC-011	2975000.00

Result 1

Output

Action Output

#	Time	Action	Message
1	19:18:17	CALL registrar_venta(1, 1, 1, @mensaje, ...	4 row(s) affected
2	19:18:39	SELECT @mensaje AS Mensaje, @total ...	1 row(s) returned

```
1   CALL consultar_stock_producto('P001', @nombre, @stock, @precio);
2
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:18:17	CALL registrar_venta(1, 1, 1, @mensaje, ...	4 row(s) affected	0.000 sec
2	19:18:39	SELECT @mensaje AS Mensaje, @total ...	1 row(s) returned	0.000 sec / 0.000 sec
3	19:19:15	CALL consultar_stock_producto('P001', ...	1 row(s) affected	0.000 sec

```
1 •  SELECT @nombre AS Nombre, @stock AS Stock, @precio AS Precio;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Nombre	Stock	Precio
Portatil HP Pavilion	14	2500000.00

Result 2 | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:18:17	CALL registrar_venta(1, 1, 1, @mensaje, ...	4 row(s) affected	0.000 sec
2	19:18:39	SELECT @mensaje AS Mensaje, @total ...	1 row(s) returned	0.000 sec / 0.000 sec
3	19:19:15	CALL consultar_stock_producto('P001', ...	1 row(s) affected	0.000 sec
4	19:19:39	SELECT @nombre AS Nombre, @stock ...	1 row(s) returned	0.000 sec / 0.000 sec

```
1  CALL calcular_ganancia_periodo('2024-09-01', '2024-09-30', @ganancia);
2
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	19:18:17	CALL registrar_venta(1, 1, 1, @mensaje, ...	4 row(s) affected	0.000 sec
2	19:18:39	SELECT @mensaje AS Mensaje, @total ...	1 row(s) returned	0.000 sec / 0.000 sec
3	19:19:15	CALL consultar_stock_producto('P001', ...	1 row(s) affected	0.000 sec
4	19:19:39	SELECT @nombre AS Nombre, @stock ...	1 row(s) returned	0.000 sec / 0.000 sec
5	19:20:02	CALL calcular_ganancia_periodo('2024-0...	1 row(s) affected	0.000 sec

```
1
2 •  SELECT @ganancia AS Ganancia_Total;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

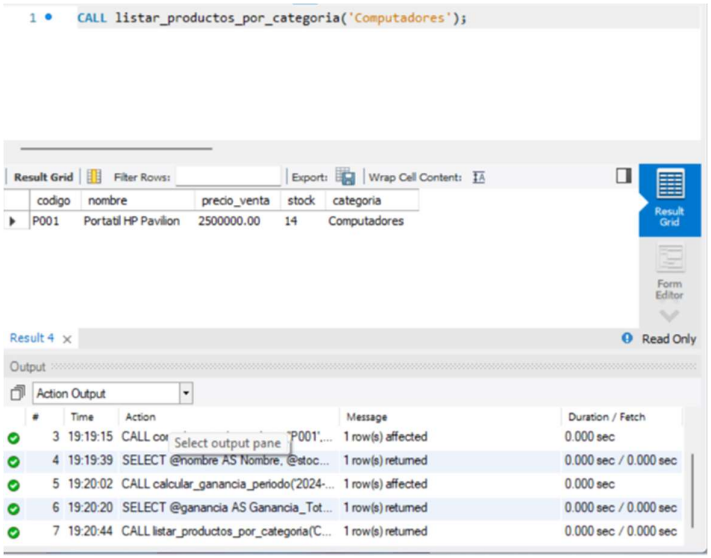
Ganancia_Total
3300000.00

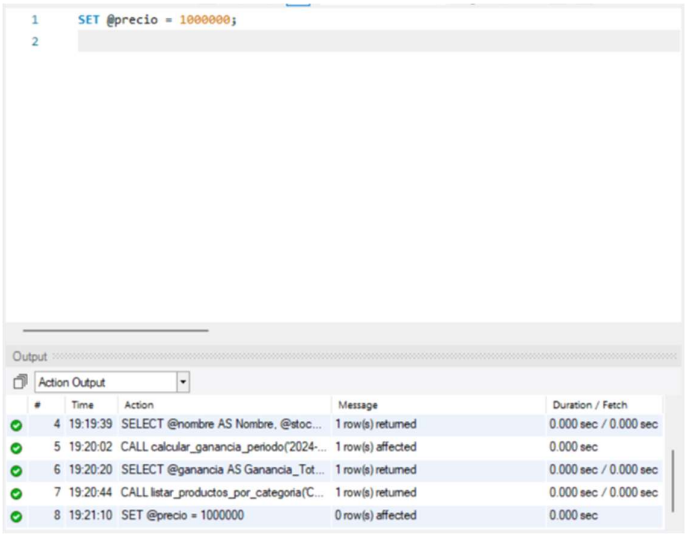
Result 3 | Read Only

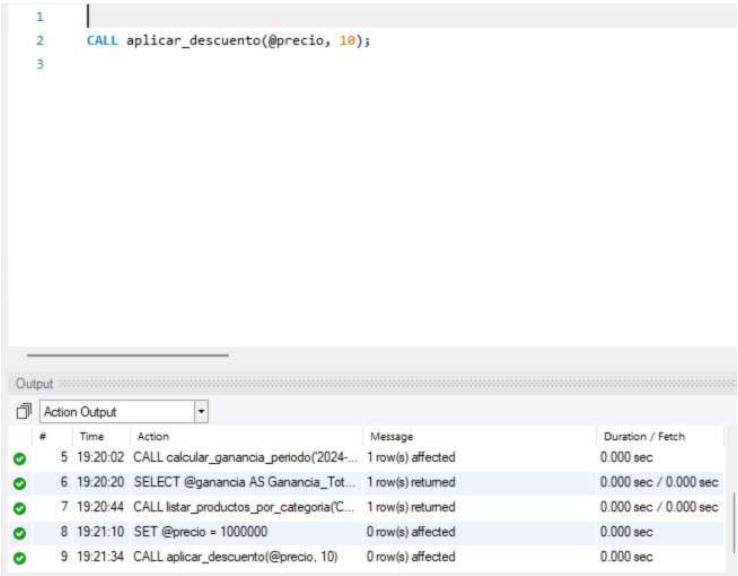
Output

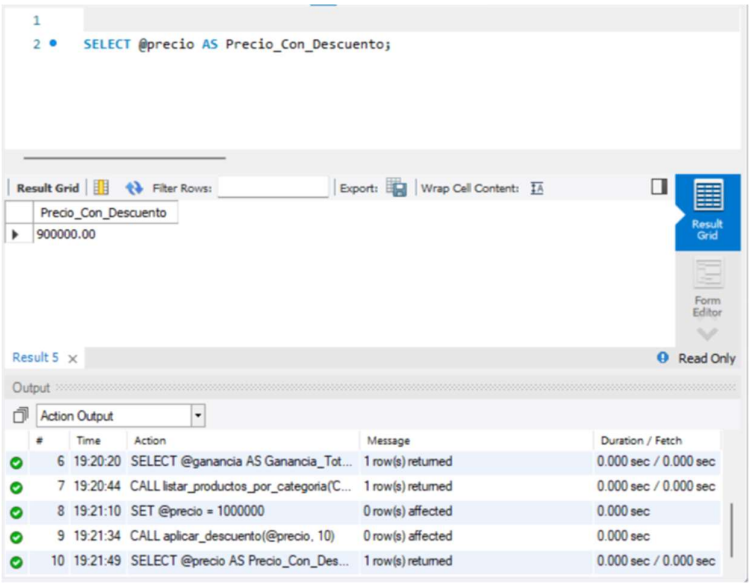
Action Output

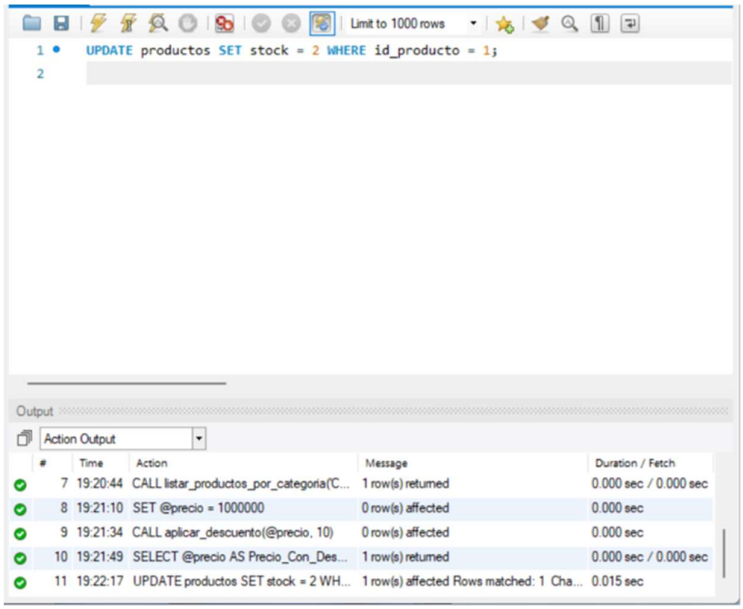
#	Time	Action	Message	Duration / Fetch
2	19:18:39	SELECT @mensaje AS Mensaje, @tota...	1 row(s) returned	0.000 sec / 0.000 sec
3	19:19:15	CALL consultar_stock_producto('P001',...	1 row(s) affected	0.000 sec
4	19:19:39	SELECT @nombre AS Nombre, @stoc...	1 row(s) returned	0.000 sec / 0.000 sec
5	19:20:02	CALL calcular_ganancia_periodo('2024-...	1 row(s) affected	0.000 sec
6	19:20:20	SELECT @ganancia AS Ganancia_Tot...	1 row(s) returned	0.000 sec / 0.000 sec


CALL listar_productos_por_categoria('Computadores');
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
codigo nombre precio_venta stock categoria
P001 Portatil HP Pavilion 2500000.00 14 Computadores
Result Grid
Form Editor
Result 4
Read Only
Output
Action Output
Time Action Message Duration / Fetch
3 19:19:15 CALL cor 'P001',... 1 row(s) affected 0.000 sec
4 19:19:39 SELECT @nombre AS Nombre, @stoc... 1 row(s) returned 0.000 sec / 0.000 sec
5 19:20:02 CALL calcular_ganancia_periodo('2024-... 1 row(s) affected 0.000 sec
6 19:20:20 SELECT @ganancia AS Ganancia_Tot... 1 row(s) returned 0.000 sec / 0.000 sec
7 19:20:44 CALL listar_productos_por_categoria('C... 1 row(s) returned 0.000 sec / 0.000 sec
Select output pane


SET @precio = 1000000;
Output
Action Output
Time Action Message Duration / Fetch
4 19:19:39 SELECT @nombre AS Nombre, @stoc... 1 row(s) returned 0.000 sec / 0.000 sec
5 19:20:02 CALL calcular_ganancia_periodo('2024-... 1 row(s) affected 0.000 sec
6 19:20:20 SELECT @ganancia AS Ganancia_Tot... 1 row(s) returned 0.000 sec / 0.000 sec
7 19:20:44 CALL listar_productos_por_categoria('C... 1 row(s) returned 0.000 sec / 0.000 sec
8 19:21:10 SET @precio = 1000000 0 row(s) affected 0.000 sec


CALL aplicar_descuento(@precio, 10);
Output
Action Output
Time Action Message Duration / Fetch
5 19:20:02 CALL calcular_ganancia_periodo('2024-... 1 row(s) affected 0.000 sec
6 19:20:20 SELECT @ganancia AS Ganancia_Tot... 1 row(s) returned 0.000 sec / 0.000 sec
7 19:20:44 CALL listar_productos_por_categoria('C... 1 row(s) returned 0.000 sec / 0.000 sec
8 19:21:10 SET @precio = 1000000 0 row(s) affected 0.000 sec
9 19:21:34 CALL aplicar_descuento(@precio, 10) 0 row(s) affected 0.000 sec


SELECT @precio AS Precio_Con_Descuento;
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
Precio_Con_Descuento
900000.00
Result Grid
Form Editor
Result 5
Read Only
Output
Action Output
Time Action Message Duration / Fetch
6 19:20:20 SELECT @ganancia AS Ganancia_Tot... 1 row(s) returned 0.000 sec / 0.000 sec
7 19:20:44 CALL listar_productos_por_categoria('C... 1 row(s) returned 0.000 sec / 0.000 sec
8 19:21:10 SET @precio = 1000000 0 row(s) affected 0.000 sec
9 19:21:34 CALL aplicar_descuento(@precio, 10) 0 row(s) affected 0.000 sec
10 19:21:49 SELECT @precio AS Precio_Con_Des... 1 row(s) returned 0.000 sec / 0.000 sec


Limit to 1000 rows
UPDATE productos SET stock = 2 WHERE id_producto = 1;
Output
Action Output
Time Action Message Duration / Fetch
7 19:20:44 CALL listar_productos_por_categoria('C... 1 row(s) returned 0.000 sec / 0.000 sec
8 19:21:10 SET @precio = 1000000 0 row(s) affected 0.000 sec
9 19:21:34 CALL aplicar_descuento(@precio, 10) 0 row(s) affected 0.000 sec
10 19:21:49 SELECT @precio AS Precio_Con_Des... 1 row(s) returned 0.000 sec / 0.000 sec
11 19:22:17 UPDATE productos SET stock = 2 WH... 1 row(s) affected Rows matched: 1 Cha... 0.015 sec

```
1
2 •  SELECT * FROM alertas_stock ORDER BY id_alerta DESC LIMIT 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

id_alerta	id_producto	stock_actual	stock_minimo	fecha_alerta	mensaje
1	1	2	3	2026-09-23 00:22:17	ALERTA: Portatil HP Pavilion tien
NULL	NULL	NULL	NULL	NULL	NULL

alertas_stock 6 | Apply

Output

Action Output

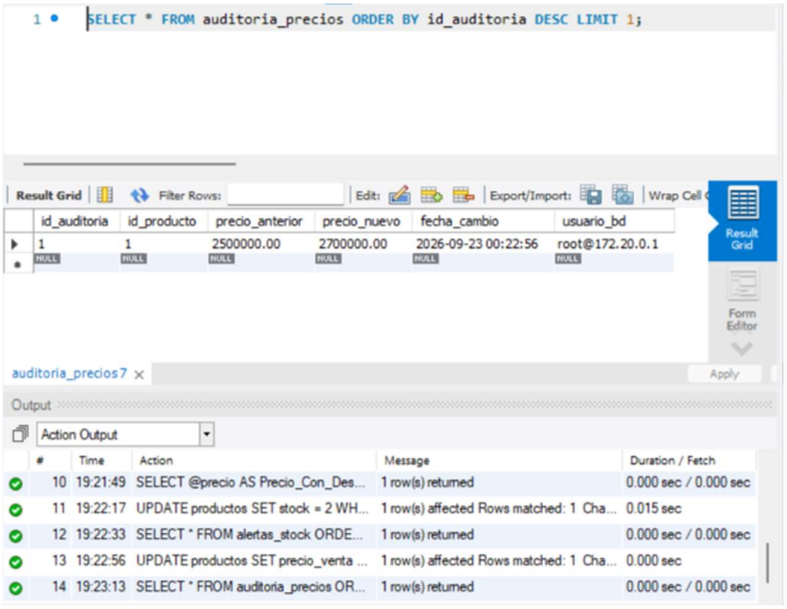
#	Time	Action	Message	Duration / Fetch
8	19:21:10	SET @precio = 1000000	0 row(s) affected	0.000 sec
9	19:21:34	CALL aplicar_descuento(@precio, 10)	0 row(s) affected	0.000 sec
10	19:21:49	SELECT @precio AS Precio_Con_Des...	1 row(s) returned	0.000 sec / 0.000 sec
11	19:22:17	UPDATE productos SET stock = 2 WH...	1 row(s) affected Rows matched: 1 Cha...	0.015 sec
12	19:22:33	SELECT * FROM alertas_stock ORDE...	1 row(s) returned	0.000 sec / 0.000 sec

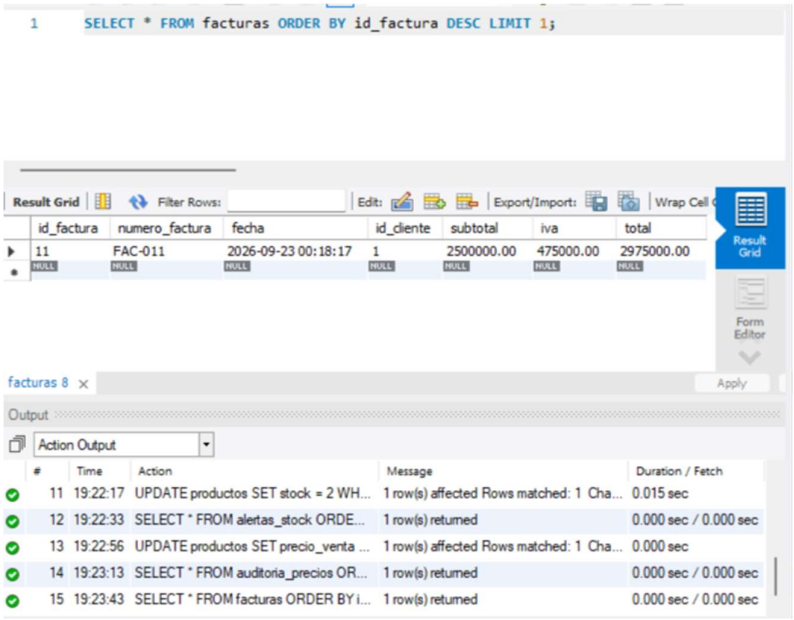
```
1 •  UPDATE productos SET precio_venta = 2700000 WHERE id_producto = 1;
2
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	19:21:34	CALL aplicar_descuento(@precio, 10)	0 row(s) affected	0.000 sec
10	19:21:49	SELECT @precio AS Precio_Con_Des...	1 row(s) returned	0.000 sec / 0.000 sec
11	19:22:17	UPDATE productos SET stock = 2 WH...	1 row(s) affected Rows matched: 1 Cha...	0.015 sec
12	19:22:33	SELECT * FROM alertas_stock ORDE...	1 row(s) returned	0.000 sec / 0.000 sec
13	19:22:56	UPDATE productos SET precio_venta ...	1 row(s) affected Rows matched: 1 Cha...	0.000 sec





Se evidencia la ejecución exitosa de los 5 procedimientos almacenados y los 3 triggers, con sus respectivos resultados.

7. CONCLUSIONES Y TRABAJO FUTURO

7.1 Aprendizajes Obtenidos

Durante el desarrollo de este avance del proyecto, se consolidaron los siguientes aprendizajes:

1. **Modelado de datos:** La importancia de normalizar correctamente una base de datos hasta 3FN para evitar redundancia y anomalías en las operaciones.
2. **Procedimientos almacenados:** La utilidad de encapsular lógica de negocio en la base de datos para reducir el tráfico de red y garantizar la consistencia de las operaciones.
3. **Triggers:** El valor de automatizar tareas de validación, auditoría y actualización para mantener la integridad de los datos sin intervención manual.
4. **Herramientas profesionales:** La experiencia de trabajar con un entorno profesional completo que incluye WSL, Docker, MariaDB, phpMyAdmin y MySQL Workbench.

8. REFERENCIAS BIBLIOGRÁFICAS

Cano Sánchez, J. J. (2026). *Guía de Configuración Docker, MySQL, PHP*. COTECNOVA.

Cano Sánchez, J. J. (2026). *Procedimientos Almacenados*. COTECNOVA.

Cano Sánchez, J. J. (2026). *Joins*. COTECNOVA.

Cano Sánchez, J. J. (2026). *Triggers*. COTECNOVA.